

Swiss Post Voting System

Verifier specification

Swiss Post*

Version 0.9.1

Abstract

The Swiss Post Voting System allows citizens to vote remotely in a secure and verifiable manner. Independent auditors can check that the system worked correctly. For that means, the system generates verifiable cryptographic evidence. The auditor reviews the provided evidence using a verifier software. The necessary verifications correspond to four algorithms: VerifyConfigPhase, VerifyVotingPhase, VerifyOnlineTally, and VerifyOfflineTally. This document details the verifications that the verifier software has to implement. At the same time, it serves as a detailed specification of the Swiss Post verifier, which is an open-source software application to verify the Swiss Post Voting System. Therefore, this document serves as a manual for developing an independent verifier software and validating or extending the Swiss Post verifier.

* Copyright 2021 Swiss Post Ltd.

Revision chart

Version	Description	Author	Reviewer	Date
0.9	First published version	OE	XM, TH	2021-09-01
0.9.1	See change log for version 0.9.1	OE, HR	XM, JS, TH	2021-10-15

Contents

Symbols	4
1 Introduction	6
1.1 The Role of the Verifier	7
1.2 Verification Categories	9
1.3 Conventions	9
2 The Verifier in the Swiss Post Voting System	10
2.1 Verification Blocks	10
2.2 Election Event Context	10
2.3 Channel Security and Control Component Authenticity	11
2.4 Basic Data Types	12
3 Block 1: Pre-election Verification — VerifyConfigPhase	13
3.1 Supporting Algorithms	18
4 Block 2: Ballot Box Verification — VerifyVotingPhase	20
4.1 Consistency Verifications	26
4.1.1 SendVote Consistency	26
4.1.2 ConfirmVote Consistency	26
4.2 Supporting Algorithms	27
4.3 Secure Logs	31
5 Block 3: Mixing Decryption Verification — VerifyOnlineTally	36
5.1 Supporting Algorithms	38
5.2 Block 3: Authenticity Verifications	40
6 Block 4: Result Verification — VerifyOfflineTally	41
6.1 Supporting Algorithms	44
6.2 Block 4: Authenticity Verifications	46
References	47
List of Algorithms	48
List of Verifications	48
List of Figures	49
List of Tables	49

Symbols

$\mathbb{A}_{Base16}$	Base16 (Hex) alphabet [10]
$\mathbb{A}_{Base64}$	Base64 alphabet [10]
$\mathbb{A}_{UCS}$	Alphabet of the Universal Coded Character Set (UCS) according to ISO/IEC10646
$\mathcal{B}$	Set of possible values for a byte
$\mathcal{B}^*$	Set of byte arrays of arbitrary length
$\mathbf{bb}$	Ballot box ID
$\mathbf{bb}$	Vector of ballot box IDs of an election event
$\mathbf{ca}$	Confirmation attempt tuple
$\mathbf{ca}$	Vector of confirmation attempt tuples
$\mathbf{ca}_{\text{final}}$	Vector of final confirmation attempt tuples
$\mathbf{ca}_{\text{non-final}}$	Vector of non-final confirmation attempt tuples
$\hat{\delta}$	Number of write-in options + 1 for a specific verification card set
δ	Maximum of $\hat{\delta}$ over all verification card sets
g	Generator of the encryption group
$\mathbb{G}_q$	Set of quadratic residues modulo p of size q . The computational proof refers to this set as $\mathbb{Q}_p$
$\mathbb{H}_l$	Ciphertext domain $(= \underbrace{\mathbb{G}_q \times \cdots \times \mathbb{G}_q}_{l+1 \text{ times}})$
i	Index variable
$\mathbf{id}$	Voter index $\{0, \dots, N_E - 1\}$
j	Index over control components $\{1, 2, 3, 4\}$
k	Number of elements of a multi-recipient ElGamal key
l	Number of elements of a multi-recipient ElGamal message
$\mathbf{l}_{HB64}$	Character length of the Base64 encoded hash function output
L_{ID}	Character length of unique identifiers
L_{votes}	List of decrypted, decoded votes
μ	Maximum supported number of write-in options + 1
m	Message representing the vote
$\mathbf{m}$	List of plaintext votes
$\mathbb{N}$	Set of positive integer numbers including 0
$\mathbb{N}^*$	Set of strictly positive integer numbers
$N_{\mathbf{bb}}$	Number of ballot boxes of an election event
N_C	Number of confirmed votes
$N_{\mathbf{CA}, \text{total}}$	Number of total confirmation attempts
$N_{\mathbf{CA}, \text{final}}$	Number of final confirmation attempts
$N_{\mathbf{CA}, \text{non-final}}$	Number of non-final confirmation attempts
N_E	Number of eligible voters
N_U	Number of unconfirmed votes
N_V	Number of total votes (confirmed and unconfirmed)
$N_{V, \text{total}}$	Number of total votes across all verification card sets
n	Number of voting options

ω	Maximum supported number of voting options
ψ	Allowed number of selections
φ	Maximum supported number of selections
$\mathbb{P}$	Set of prime numbers
$\mathbf{p}$	Vector of small group members and prime
$\tilde{\mathbf{p}}$	Encoded voting options $(\tilde{p}_1, \dots, \tilde{p}_n)$, $\tilde{p}_k \in (\mathbb{G}_q \cap \mathbb{P}) \setminus g$
p	Encryption group modulus
q	Encryption group cardinality s.t. $p = 2q + 1$
$\mathbf{vc}$	Verification card
$\mathbf{vc}$	Vector of verification card IDs of an election event
$\mathbf{vc}_{\text{sent}}$	Vector of sent verification card IDs of an election event
$\mathbf{vc}_{\text{confirmed}}$	Vector of confirmed verification card IDs of an election event
$\mathbf{vcs}$	Verification card set ID
$\mathbf{vcs}$	Vector of verification card set IDs of an election event
$ x $	Bit length of the number x
$\mathbb{Z}_q$	Set of integers modulo q

1 Introduction

Switzerland has a longstanding tradition of direct democracy, allowing Swiss citizens to vote approximately four times a year on elections and referendums. In recent years, voter turnout hovered below 40 percent [4].

The vast majority of voters in Switzerland fill out their paper ballots at home and send them back to the municipality by postal mail, usually days or weeks ahead of the actual election date. Remote online voting (referred to as e-voting in this document) would provide voters with some advantages. First, it would guarantee the timely arrival of return envelopes at the municipality (especially for Swiss citizens living abroad). Second, it would improve accessibility for people with disabilities. Third, it would eliminate the possibility of an invalid ballot when inadvertently filling out the ballot incorrectly.

In the past, multiple cantons offered e-voting to a part of their electorate. Many voters would welcome the option to vote online - provided the e-voting system protects the integrity and privacy of their vote [5].

State-of-the-art e-voting systems alleviate the practical concerns of mail-in voting and, at the same time, provide a high level of security. Above all, they must display three properties [13]:

- Individual verifiability: allow a voter to convince herself that the system correctly registered her vote
- Universal verifiability: allow an auditor to check that the election outcome corresponds to the registered votes
- Vote secrecy: do not reveal a voter's vote to anyone

Following these principles, the Federal Chancellery defined stringent requirements for e-voting systems. The current draft of the Ordinance on Electronic Voting (VEleS - Verordnung über die elektronische Stimmabgabe) and its technical annex (VEleS annex)[2] describes these requirements.

Swiss democracy deserves an e-voting system with excellent security properties. In response to the release of an earlier version of the documentation and the source code of the e-voting system, multiple researchers published attacks, highlighted vulnerabilities, and suggested improvements[6, 9, 11, 14]. Swiss Post is thankful to all security researchers for their contributions and the opportunity to improve the system's security guarantees. We look forward to actively engaging with academic experts and the hacker community to maximize public scrutiny of the Swiss Post Voting System.

1.1 The Role of the Verifier

A verifiable e-voting system requires a verifiable process and a verification software—the *verifier*—to verify the cryptographic evidence using data published on a private or public bulletin board [8]. The specification and development of the verifier should go hand in hand with the e-voting solution, and the verifier challenges and extensively tests a protocol run [7].

Therefore, the current draft of the Ordinance on Electronic Voting (VEleS)[2] defines the role of the auditors and their technical aid.

[VEleS Art 2.h]: *auditor* means a person who checks on behalf of the canton that the ballot is correctly conducted.

[VEleS Art 5.b]: The auditors evaluate the proof in an observable procedure; to do so, they must use *technical aids* that are independent of and isolated from the rest of the system.

For the rest of the document, we will no longer distinguish between auditors and their technical aid; we refer to *the verifier* as both the auditor and the software used by this auditor and assume that the auditor and the technical aid are trustworthy. There may be multiple auditors, and an auditor may use various technical aids. The technical annex of the Ordinance on Electronic Voting states that at least one of the auditors and one of the technical aids is trustworthy.

[VEleS Annex 2.9.2.2]: The following system participants may be considered trustworthy

- [...]
- one auditor in any group, leaving open which auditor it is
- one technical aid from a trustworthy auditor, leaving open which aid it is.

In principle, everybody could become an auditor and could check an election event's proofs and data by themselves. The auditors "represent" voters in the sense of universal verifiability as elaborated in the Federal Chancellery's explanatory report [3].

[Explanatory Report, Sec 5.2.1]: The use of auditors promotes transparency. Voters should be able to assume that auditors will draw attention to possible irregularities.

[Explanatory Report, Sec 5.2.1]: With universal verifiability, manipulations in the infrastructure can be detected. Unlike individual verifiability, it does not necessarily have to be offered to voters. Instead, auditors can be employed to apply universal verifiability.

[VEleS Art 2.i]: Infrastructure means hardware, software [...], network elements, premises, services and equipment of any nature at any operating bodies that are required for the secure operation of electronic voting.

The auditors are subject to the following requirement:

[VEleS Annex 8.13]: The auditors should be suitably informed about and trained in the processes that determine the accuracy of the result, the preservation of voting secrecy and the avoidance of premature results (for example key generation, printing the voting papers, decryption and tallying). They must be able to understand the essential aspects of the processes and their significance.

However, the exact selection of auditors is a cantonal responsibility and out of the scope of this document:

[VEleS Art. 15]: Tasks of the body responsible at cantonal level.
The canton shall appoint a body that bears overall responsibility, and for the following tasks in particular. [...]
h. appointing and instructing the auditors.

Swiss Post releases its verifier under a permissive *open-source license*: strengthening the independence and trustworthiness of the verifiability of the system and complying with the Ordinance:

[VEleS Annex 3.17]: The software for the auditors' technical aids must be obtained from a different system developer from the one who developed the main part of the software for the other system components. The publication of the software for the technical aid under a licence that meets the criteria for open source software may justify an exception.

1.2 Verification Categories

Haenni et al. highlight five categories of verifications that a verifier is supposed to perform [8]. Table 1 shows these categories.

Category	Description
Evidence	Are the cryptographic evidence contained in the election data all valid? Do they provide the necessary evidence to infer the correctness of corresponding protocol steps?
Authenticity	Can the data elements be linked unambiguously to the party authorized to create them?
Consistency	Are related data items consistent to each other?
Integrity	Do all data elements correspond to the specification? Are they all within the specified ranges?
Completeness	Do the data elements allow a complete verification chain?

Tab. 1: Verification categories and their description.

This document focuses on the verification of the cryptographic *evidence*. Moreover, we designate important *authenticity checks* that the verifier must perform. When necessary, the verifier specification also highlights important *consistency* checks. Since we use a mathematically precise pseudo-code specifications, most of the *integrity* checks are implicit. Furthermore, we base our verifier specification on the computational proof of complete verifiability and privacy [12], thereby making sure that our verifications are *complete* and ensure the necessary security objectives.

1.3 Conventions

We use the following conventions throughout the document.

- Display algorithms in font without serif and **vectors** in **boldface**.
- Provide the domain of input and output variables.
- Distinguish context (invariant) and input variables (different for each invocation).
- Use the terms public key and secret key (instead of private key) and abbreviate them with pk and sk .
- Use 0-based indexing.
- Define sets for ranges, e.g. for $i \in [0, n)$.

2 The Verifier in the Swiss Post Voting System

2.1 Verification Blocks

The Swiss Post Voting System consists of three phases: Configuration, Voting, and Tally. Each phase has a corresponding verification algorithm, and for each verification algorithm, we define a corresponding verification block. A verification block groups the different verifications that the verifier performs. Table 2 highlights the four verification blocks.

Block	Phase	Algorithm	Section
Pre-election verification	Configuration	VerifyConfigPhase	3
Ballot box verification	Voting	VerifyVotingPhase	4
Mixing decryption verification	Tally	VerifyOnlineTally	5
Result verification	Tally	VerifyOfflineTally	6

Tab. 2: Overview of the Verification Blocks.

Each building block processes files containing election-related or cryptographic information to ascertain the Swiss Post Voting System’s specific security properties and the election result’s correctness.

2.2 Election Event Context

Verifications are executed per election event, per verification card set, per ballot box, or voting card. Each election event has one or multiple verification card sets. Each verification card set has one ballot box, and each verification card set contains one or multiple eligible voters. Figure 1 highlights the relevant entities.

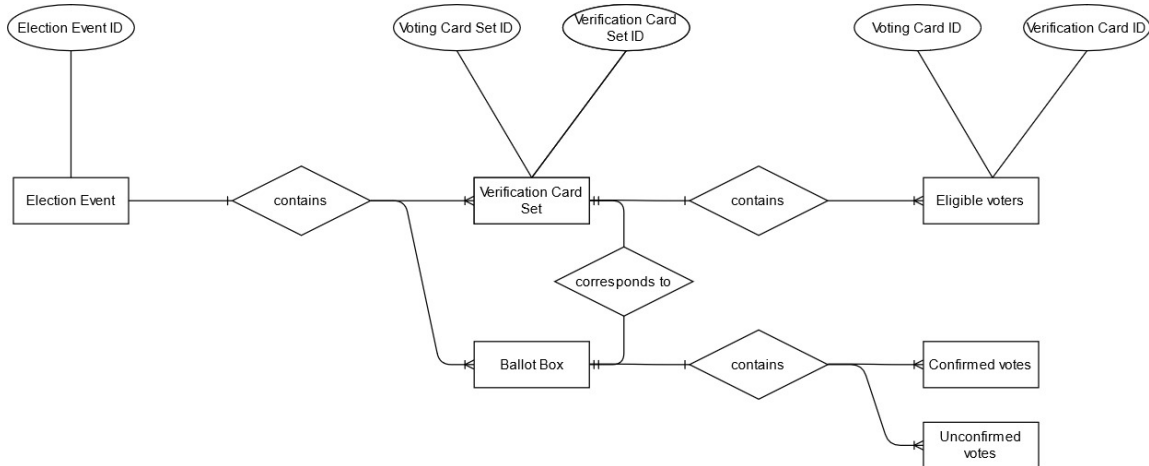


Fig. 1: Overview of the election event context. Every election event has multiple verification card sets. Each verification card set has a corresponding ballot box. A verification card set has N_E eligible voters; a ballot box contains N_C confirmed votes.

We define an election event’s vector of verification card set IDs $\mathbf{vcs} = (\mathbf{vcs}_0, \dots, \mathbf{vcs}_{N_{bb}-1})$ and vector of ballot box IDs $\mathbf{bb} = (\mathbf{bb}_0, \dots, \mathbf{bb}_{N_{bb}-1})$, where N_{bb} is the number of ballot boxes of an election event.

2.3 Channel Security and Control Component Authenticity

A meaningful verification of election event data must include a check that the protocol's run involved the actual honest components. Otherwise, the adversary could impersonate protocol participants — undermining verifiability and vote secrecy.

Recall that our trust model considers the setup component and one out of four control components trustworthy. The Ordinance's explanatory report elaborates on the term *trustworthy*.

[Explanatory Report, Sec 5.2.2]: Cryptographic protocols make it possible to reduce to a minimum the number of elements that an attacker would have to control in order to manipulate votes without being detected or violate voter secrecy. Measures to prevent an attacker from taking control of an element can therefore focus on a limited number of elements. These elements are particularly worthy of protection and, ideally, can also be protected particularly effectively. Such elements found under Numbers 2.1 and 2.2 'System participants' and 'Communication channels' are referred to as 'trustworthy'. This may seem surprising at first glance: why is an element that is particularly worthy of protection called 'trustworthy'? The reason lies in the fact that cryptographic protocols are not aimed at protecting those elements. The designation 'trustworthy' signals to authors and readers of the document in which the cryptographic protocol is specified that they do not need to worry about possible attacks in which an attacker takes control of these elements. By being trustworthy, system participants *refuse* to cooperate with an attacker. The protocol must be defined in such a way that, as long as the trustworthy system participants adhere to the protocol, the attacker will not succeed even if they bring the remaining non-trustworthy system participants under control. The use of the term is based on the literature.

Moreover, the model foresees a secure and authentic communication channel between the setup component and the verifier. On the other hand, there is no trusted communication channel from the control components to the verifier: they transmit all information through insecure communication channels and untrusted system components. Therefore, the control components digitally sign their data and link them to a specific election event ID to prevent replay attacks. The verifier's authenticity checks verify the digital signature, and the consistency checks ensure that all data items relate to the correct election event ID.

Digital signatures imply that the verifier must know that a public key X belongs to control component A. Every control component designates a responsible person that transmits the certificate's hash out-of-band to the auditor during a certificate ceremony. Figure 2 highlights the verification procedure.

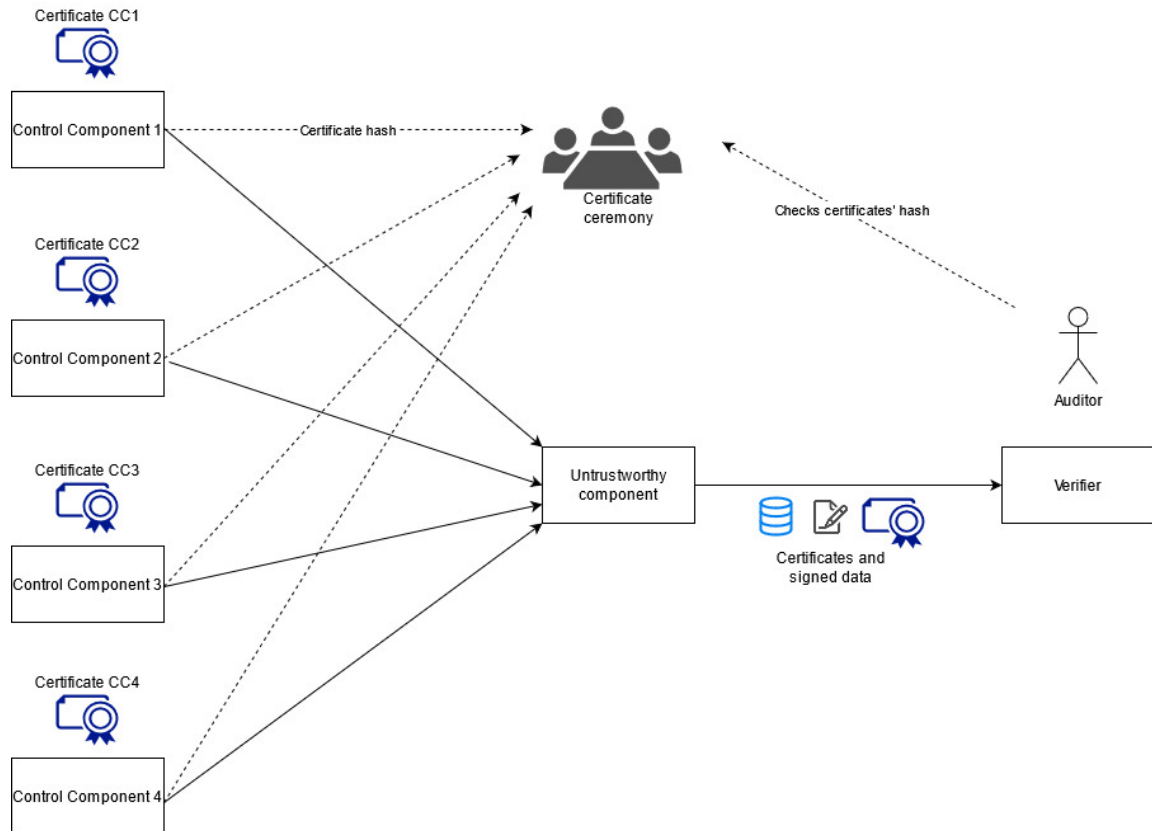


Fig. 2: The control components sign their data and send the certificate, signature, and data via an untrusted component to the verifier. The auditors ensure the certificate's authenticity with a ceremonial procedure.

The certificate ceremony allows the auditors to verify that the public key corresponds to the authorized party and ensure the control components' authenticity.

2.4 Basic Data Types

The [Crypto-Primitives specification](#) details how we represent, convert, and operate on basic data types such as bytes, integers, strings, and arrays.

3 Block 1: Pre-election Verification — VerifyConfigPhase

The pre-election verification block verifies the correct setup of the election event. Namely, this block demonstrates that the Swiss Post Voting System configured cryptographically sound parameters, associated each voting option to a prime number, generated the correct number of voting cards, and signed the relevant configuration information.

A positive verification result of this block indicates to the auditor that the configuration allows the system to perform reasonably secure operations and to provide meaningful cryptographic evidence. In contrast, failed verifications in that block cast doubts about the system's security properties: secure encryption of votes, unforgeability of digital signatures, and the soundness of zero-knowledge proofs rely on the proper configuration of the cryptographic parameters.

Verification 1.01 VerifyEncryptionParameters

Context:

Input:

Provided group modulus $\hat{p} \in \mathbb{P}$
 Provided group cardinality $\hat{q} \in \mathbb{P}$ s.t. $p = 2q + 1$
 Provided group generator $\hat{g} \in \mathbb{G}_q$
 $seed \in \mathbb{A}_{UCS}^*$

Operation:

```

1:  $(p, q, g) \leftarrow \text{GetEncryptionParameters}(seed)$  ▷ See crypto primitives specification
2: if  $(p = \hat{p}) \wedge (q = \hat{q}) \wedge (g = \hat{g})$  then
3:   return  $\top$ 
4: else
5:   return  $\perp$ 
6: end if
    
```

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

The verifier checks that the small primes—excluding the generator g —of a mathematical group $\mathbb{G}_q$ encode the voting options. Since the voting client multiplies all selected voting options prior to encryption, the verifier ensures that the maximum product of φ voting options does not exceed p to prevent modulo overflow.

Verification 1.02 VerifySmallPrimeGroupMembers

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
- Group generator $g \in \mathbb{G}_q$
- Maximum supported number of voting options $\omega \in \mathbb{N}^*$

Input:

The small prime group members in ascending order $\mathbf{p} = (p_0, \dots, p_{\omega-1}) \in (\mathbb{G}_q \cap \mathbb{P}) \setminus \{2, 3\}$

Operation:

- 1: $\mathbf{p}' \leftarrow \text{GetSmallPrimeGroupMembers}(p, q, g, \omega)$ ▷ See crypto primitives specification
 - 2: **if** $\mathbf{p}' = \mathbf{p}$ **then**
 - 3: **return** $\top$
 - 4: **else**
 - 5: **return** $\perp$
 - 6: **end if**
-

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

Verification 1.03 VerifyVotingOptions

Context:

- Group modulus $p \in \mathbb{P}$
- Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
- Group generator $g \in \mathbb{G}_q$
- Maximum supported number of voting options $\omega \in \mathbb{N}^*$
- Maximum supported number of selections $\varphi \in \mathbb{N}^*$

Input:

- Small prime group members in ascending order $\mathbf{p} = (p_0, \dots, p_{\omega-1})$, $p_i \in (\mathbb{G}_q \cap \mathbb{P}) \setminus \{2, 3\}$
- Encoded voting options $\tilde{\mathbf{p}} = (\tilde{p}_0, \dots, \tilde{p}_{n-1})$, $\tilde{p}_i \in (\mathbb{G}_q \cap \mathbb{P}) \setminus \{2, 3\}$

Ensure:

- $\varphi \leq \omega$
 - $0 < n \leq \omega$
-

Operation:

- 1: $\mathbf{p}' \leftarrow \{p_0, \dots, p_{n-1}\}$
 - 2: $\tilde{\mathbf{p}}' \leftarrow \{\tilde{\mathbf{p}}\}$
 - 3: **if** $\mathbf{p}' = \tilde{\mathbf{p}}'$ **then**
 - 4: $\text{verifA} \leftarrow \top$
 - 5: **else**
 - 6: $\text{verifA} \leftarrow \perp$
 - 7: **end if**
 - 8: $\text{verifB} \leftarrow \prod_{i=(\omega-\varphi)}^{\omega-1} p_i < p$ $\triangleright$ The product of the φ last primes (the largest possible encoded vote) must be smaller than p
 - 9: **return** $\text{verifA} \wedge \text{verifB}$
-

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

The subsequent algorithms check that the control components correctly exponentiated the encrypted partial Choice Return Codes $\mathbf{pCC}_{\text{id}}$ and the encrypted Confirmation Key $\mathbf{CK}_{\text{id}}$. The verifier checks that *all* exponentiation proofs of *all* verification card sets validate. The verifier expects that every verification card set contains at least one verification card.

Verification 1.21 VerifyEncryptedPCCExponentiationProofs

Input:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Vector of verification card set IDs $\mathbf{vcs} = (\mathbf{vcs}_0, \dots, \mathbf{vcs}_{N_{bb}-1}) \in ((\mathbb{A}_{Base16})^{L_{ID}})^{N_{bb}}$

Operation:

```

1: for  $i \in [0, N_{bb})$  do
2:   if  $\mathbf{vcs}_i$  contains no proofs then
3:      $\mathbf{vcsEncryptedPCCVerif}_i \leftarrow \perp$  ▷ Missing proofs correspond to an error case
4:   else
5:      $\mathbf{vcsEncryptedPCCVerif}_i \leftarrow \text{VerifyEncryptedPCCExponentiationProofsVerificationCardSet}$ 
      (Inputs for verification card set  $\mathbf{vcs}_i$ ) ▷ See Algorithm 3.1
6:   end if
7: end for
8: if  $\mathbf{vcsEncryptedPCCVerif}_i \forall i$  then
9:   return  $\top$ 
10: else
11:   return  $\perp$ 
12: end if

```

Output:

The result of the verification: $\top$ if the verification is successful for *all* verification card sets,
 $\perp$ otherwise.

Verification 1.22 VerifyEncryptedCKExponentiationProofs

Input:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Vector of verification card set IDs $\mathbf{vcs} = (\mathbf{vcs}_0, \dots, \mathbf{vcs}_{N_{bb}-1}) \in ((\mathbb{A}_{Base16})^{L_{ID}})^{N_{bb}}$

Operation:

```
1: for  $i \in [0, N_{bb})$  do
2:   if  $\mathbf{vcs}_i$  contains no proofs then
3:      $\mathbf{vcsEncryptedCKVerif}_i \leftarrow \perp$  ▷ Missing proofs correspond to an error case
4:   else
5:      $\mathbf{vcsEncryptedCKVerif}_i \leftarrow \text{VerifyEncryptedCKExponentiationProofsVerificationCardSet}$ 
      (Inputs for verification card set  $\mathbf{vcs}_i$ ) ▷ See Algorithm 3.2
6:   end if
7: end for
8: if  $\mathbf{vcsEncryptedCKVerif}_i \forall i$  then
9:   return  $\top$ 
10: else
11:   return  $\perp$ 
12: end if
```

Output:

The result of the verification: $\top$ if the verification is successful for *all* verification card sets,
 $\perp$ otherwise.

3.1 Supporting Algorithms

The verifications in block 1 rely on the following algorithms.

Algorithm 3.1 VerifyEncryptedPCCExponentiationProofsVerificationCardSet

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
 Group generator $g \in \mathbb{G}_q$
 Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$
 Verification Card Set ID $\mathbf{vcs} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Input:

Vector of verification card IDs $\mathbf{vc} = (\mathbf{vc}_0, \dots, \mathbf{vc}_{N_E-1}) \in (\mathbb{A}_{Base16})^{L_{ID} \times N_E}$
 Encrypted, hashed partial Choice Return Codes $\mathbf{c}_{PCC} \in (\mathbb{G}_q \times \mathbb{G}_q^n)^{N_E}$
 Voter Choice Return Code Generation public keys $(\mathbf{K}_1, \mathbf{K}_2, \mathbf{K}_3, \mathbf{K}_4) \in (\mathbb{G}_q^{N_E})^4$
 Exponentiated, encrypted, hashed partial Choice Return Codes
 $(\mathbf{c}_{expPCC,1}, \mathbf{c}_{expPCC,2}, \mathbf{c}_{expPCC,3}, \mathbf{c}_{expPCC,4}) \in ((\mathbb{G}_q \times \mathbb{G}_q^n)^{N_E})^4$
 Proofs of correct exponentiation $(\boldsymbol{\pi}_{expPCC,1}, \boldsymbol{\pi}_{expPCC,2}, \boldsymbol{\pi}_{expPCC,3}, \boldsymbol{\pi}_{expPCC,4}) \in ((\mathbb{Z}_q \times \mathbb{Z}_q)^{N_E})^4$

Operation:

```

1: for  $j \in [1, 4]$  do
2:   for  $\mathbf{id} \in [0, N_E)$  do
3:      $\mathbf{g} \leftarrow (g, \mathbf{c}_{PCC,\mathbf{id}})$  ▷ The bases
4:      $\mathbf{y} \leftarrow (\mathbf{K}_{j,\mathbf{id}}, \mathbf{c}_{expPCC,j,\mathbf{id}})$  ▷ The exponentiations
5:      $\mathbf{i}_{aux} \leftarrow (\mathbf{ee}, \mathbf{vc}_{\mathbf{id}}, \text{"GenEncLongCodeShares"}, \text{IntegerToString}(j))$ 
6:      $\text{exponentiationVerif}_{j,\mathbf{id}} \leftarrow \text{VerifyExponentiation}(\mathbf{g}, \mathbf{y}, \boldsymbol{\pi}_{expPCC,j,\mathbf{id}}, \mathbf{i}_{aux})$  ▷ See crypto
    primitives specification
7:   end for
8: end for
9: if  $\text{exponentiationVerif}_{j,\mathbf{id}} \forall \{j, \mathbf{id}\}$  then
10:  return  $\top$ 
11: else
12:  return  $\perp$ 
13: end if
```

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* verification card set, $\perp$ otherwise.

Algorithm 3.2 VerifyEncryptedCKExponentiationProofsVerificationCardSet

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
 Group generator $g \in \mathbb{G}_q$
 Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$
 Verification Card Set ID $\mathbf{vcs} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Input:

Vector of verification card IDs $\mathbf{vc} = (\mathbf{vc}_0, \dots, \mathbf{vc}_{N_E-1}) \in (\mathbb{A}_{Base16})^{L_{ID} \times N_E}$
 Encrypted, hashed Confirmation Key $\mathbf{c}_{ck} \in (\mathbb{G}_q \times \mathbb{G}_q)^{N_E}$
 Voter Vote Cast Return Code Generation public keys $(\mathbf{Kc}_1, \mathbf{Kc}_2, \mathbf{Kc}_3, \mathbf{Kc}_4) \in (\mathbb{G}_q^{N_E})^4$
 Exponentiated, encrypted, hashed Confirmation Key $(\mathbf{c}_{expCK,1}, \mathbf{c}_{expCK,2}, \mathbf{c}_{expCK,3}, \mathbf{c}_{expCK,4}) \in ((\mathbb{G}_q \times \mathbb{G}_q)^{N_E})^4$
 Proofs of correct exponentiation $(\boldsymbol{\pi}_{expCK,1}, \boldsymbol{\pi}_{expCK,2}, \boldsymbol{\pi}_{expCK,3}, \boldsymbol{\pi}_{expCK,4}) \in ((\mathbb{Z}_q \times \mathbb{Z}_q)^{N_E})^4$

Operation:

```

1: for  $j \in [1, 4]$  do
2:   for  $\mathbf{id} \in [0, N_E)$  do
3:      $\mathbf{g} \leftarrow (g, \mathbf{c}_{ck, \mathbf{id}})$  ▷ The bases
4:      $\mathbf{y} \leftarrow (\mathbf{Kc}_{j, \mathbf{id}}, \mathbf{c}_{expCK, j, \mathbf{id}})$  ▷ The exponentiations
5:      $\mathbf{i}_{aux} \leftarrow (\mathbf{ee}, \mathbf{vc}_{\mathbf{id}}, \text{"GenEncLongCodeShares"}, \text{IntegerToString}(j))$ 
6:      $\text{exponentiationVerif}_{j, \mathbf{id}} \leftarrow \text{VerifyExponentiation}(\mathbf{g}, \mathbf{y}, \boldsymbol{\pi}_{expCK, j, \mathbf{id}}, \mathbf{i}_{aux})$  ▷ See crypto
       exponentiationVerif  $\leftarrow$  primitives specification
7:   end for
8: end for
9: if  $\text{exponentiationVerif}_{j, \mathbf{id}} \forall \{j, \mathbf{id}\}$  then
10:  return  $\top$ 
11: else
12:  return  $\perp$ 
13: end if

```

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* verification card set, $\perp$ otherwise.

4 Block 2: Ballot Box Verification — VerifyVotingPhase

The ballot box verification block verifies the correct processing of incoming votes and confirmation attempts—which the Swiss Post Voting System eventually stores in the ballot box. For this purpose, it uses secure logs (immutable audit logs) that the independent control components generate and digitally sign. The control components cryptographically link these secure logs’ entries to create a tamper-proof immutable chain of records. Whenever a control component generates a voting card (during the configuration phase) or processes a vote (during the voting phase), it stores information in a secure log entry that an auditor can use in this block. Moreover, cross-checking the secure logs of the individual control components guarantees that they processed the same information. Section 4.3 explains the verification of secure logs.

A positive verification result of the ballot box verification block indicates to the auditor—under the trust assumption that at least one control component is *trustworthy*—that the Swiss Post Voting System correctly processed every incoming vote and confirmation attempt.

Each Return Codes control component CCR_j processed votes $N_V(\mathbf{vcs})$ votes per verification card set; both confirmed and unconfirmed votes $N_V(\mathbf{vcs}) = N_C(\mathbf{vcs}) + N_U(\mathbf{vcs})$ (see figure 1). We designate the total number of processed votes (confirmed and unconfirmed) as

$$N_{V,\text{total}} = \sum_{i=0}^{N_{bb}-1} N_V(\mathbf{vcs}_i)$$

Correspondingly, we have the vector of sent verification card IDs $\mathbf{vc}_{\text{sent}} = (\mathbf{vc}_0, \dots, \mathbf{vc}_{N_{V,\text{total}}-1})$

Verification 2.21 VerifyPCCExponentiationProofs

Input:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Vector of sent verification card IDs $\mathbf{vc}_{\text{sent}} = (\mathbf{vc}_0, \dots, \mathbf{vc}_{N_{V,\text{total}}-1}) \in ((\mathbb{A}_{Base16})^{L_{ID}})^{N_{V,\text{total}}}$

Operation:

- 1: **for** $i \in [0, N_{V,\text{total}})$ **do**
 - 2: $\text{PCCVerif}_i \leftarrow \text{VerifyPCCExponentiationProofsVC}(\text{Inputs for verification card } \mathbf{vc}_i)$ ▷ See Algorithm 4.1
 - 3: **end for**
 - 4: **if** $\text{PCCVerif}_i \forall i$ **then**
 - 5: **return** $\top$
 - 6: **else**
 - 7: **return** $\perp$
 - 8: **end if**
-

Output:

The result of the verification: $\top$ if the verification is successful for *all sent* votes, $\perp$ otherwise.

Then, the verifier checks that each Choice Return Code is extractable by verifying that each long Choice Return Code $1CC_{id,i}$ has a corresponding entry in the Return Codes Mapping table CMtable.

Verification 2.22 VerifyExtractableCC

Input:

Election event ID $ee \in (\mathbb{A}_{Base16})^{L_{ID}}$

Vector of sent verification card IDs $\mathbf{vc}_{sent} = (vc_0, \dots, vc_{N_{V,total}-1}) \in ((\mathbb{A}_{Base16})^{L_{ID}})^{N_{V,total}}$

Operation:

```

1: for  $i \in [0, N_{V,total})$  do
2:    $extractableCC_i \leftarrow \text{VerifyExtractableCCVerificationCard}(\text{Inputs for verification card } vc_i)$ 
    $\triangleright$  See Algorithm 4.2
3: end for
4: if  $extractableCC_i \forall i$  then
5:   return  $\top$ 
6: else
7:   return  $\perp$ 
8: end if

```

Output:

The result of the verification: $\top$ if the Choice Return Codes are extractable for *all* sent votes, $\perp$ otherwise.

Verification 2.23 VerifyCKExponentiationProofs
$$\text{Vector of confirmation attempt tuples } \mathbf{ca} = (\mathbf{ca}_0, \dots, \mathbf{ca}_{N_{\text{CA}, \text{total}}-1}) \in \left((\mathbb{A}_{\text{Base16}})^{L_{\text{ID}}}, [0, 5) \right)^{N_{\text{CA}, \text{total}}}$$

```

1: for  $i \in [0, N_{\text{CA}, \text{total}})$  do
2:    $\text{CKVerif}_i \leftarrow \text{VerifyCKExponentiationProofsConfirmationAttempt}$ 
   (Inputs for confirmation attempt  $\text{ca}_i$ ) ▷ See Algorithm 4.3
3: end for
4: if  $\text{CKVerif}_i \forall i$  then
5:   return  $\top$ 
6: else
7:   return  $\perp$ 
8: end if

```

The result of the verification: $\top$ if the verification is successful for *all* confirmation attempts, $\perp$ otherwise.

In contrast to the long Choice Return Codes (see verification 2.22), not every long Vote Cast Return Code is extractable: a voter might have entered one or multiple times an incorrect Ballot Casting Key BCK_{id} . Therefore, the verifier's goal is a different one: if the voter entered the correct Ballot Casting Key BCK_{id} , the voting client generated the correct Confirmation Key CK_{id} , and the control components performed the exponentiation correctly (see verification 2.23), an extractable Vote Cast Return Code signifies that the vote was rightly confirmed.

Only the final confirmation attempt of each verification card vc_{id} can correspond to a successful confirmation. If a voter has three confirmation attempts, only the third can correspond to a valid confirmation. The consistency verifications in section 4.1.2 ensure that all control components have a consistent view of each processed confirmation attempt.

Therefore, we define the vector of non-final confirmation attempts $\mathbf{ca}_{\text{non-final}} \subseteq \mathbf{ca}$ and the vector of final confirmation attempts $\mathbf{ca}_{\text{final}} \subseteq \mathbf{ca}$. It holds that $\mathbf{ca}_{\text{non-final}} \cap \mathbf{ca}_{\text{final}} = \mathbf{ca}$.

The verifier checks that *all* non-final confirmation attempts are *not* extractable in the Return Codes Mapping table CMtable .

Verification 2.24 VerifyNonFinalConfirmationAttempts

Input:

Election event ID $\text{ee} \in (\mathbb{A}_{\text{Base16}})^{\text{LID}}$

Vector of non-final confirmation attempt tuples $\mathbf{ca}_{\text{non-final}} \in \left((\mathbb{A}_{\text{Base16}})^{\text{LID}}, [0, 5) \right)^{N_{\text{CA,non-final}}}$

Operation:

```

1: for  $i \in [0, N_{\text{CA,non-final}})$  do
2:    $\text{nonFinalConfirmation}_i \leftarrow \text{VerifyExtractableVCCConfirmationAttempt}$ 
   (Inputs for confirmation attempt  $\mathbf{ca}_{\text{non-final},i}$ ) ▷ See Algorithm 4.4
3: end for
4: if  $\text{nonFinalConfirmation}_i \forall i$  then
5:   return  $\top$ 
6: else
7:   return  $\perp$ 
8: end if

```

Output:

The result of the verification: $\top$ if *all* non-final confirmation attempts are *not* extractable, $\perp$ otherwise.

As explained previously, some of the final confirmation attempts may not correspond to successful confirmations. The voting server maintains a list of votes with successful confirmations, designated as $\mathbf{vc}_{\text{confirmed}}$, and containing a vector of verification card IDs. The next verification checks that $\mathbf{vc}_{\text{confirmed}}$ is correct.

We define a local function `retrieveVC` that retrieves the verification card $\mathbf{vc}_i$ from a confirmation attempt tuple, i.e. $\text{retrieveVC}(\mathbf{ca}_{\text{final},i}) = \mathbf{vc}_i$ or returns the vector of verification card IDs from the vector of confirmation attempt tuples, i.e. $\text{retrieveVC}(\mathbf{ca}_{\text{final}}) = \mathbf{vc}$.

Verification 2.25 VerifyListOfVotesWithSuccessfulConfirmation

Input:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{\text{Base16}})^{\text{LID}}$

Vector of final confirmation attempt tuples $\mathbf{ca}_{\text{final}} \in ((\mathbb{A}_{\text{Base16}})^{\text{LID}}, [0, 5))^{\text{N}_{\text{CA},\text{final}}}$

Voting server's list of successful votes $\mathbf{vc}_{\text{confirmed}} \in (\mathbb{A}_{\text{Base16}})^{\text{Nc}}$

Ensure: $\mathbf{vc}_{\text{confirmed},i} \neq \mathbf{vc}_{\text{confirmed},k}, \forall i, k \in \{0, \dots, (\text{Nc} - 1)\} \wedge i \neq k$ $\triangleright$ Ensures that all verification cards in the list of successful votes are distinct

Ensure: $\text{retrieveVC}(\mathbf{ca}_{\text{final},i}) \neq \text{retrieveVC}(\mathbf{ca}_{\text{final},k}), \forall i, k \in \{0, \dots, (\text{N}_{\text{CA},\text{final}} - 1)\} \wedge i \neq k$ $\triangleright$ Ensures that all verification cards in the vector of confirmation attempts are distinct

Ensure: $\mathbf{vc}_{\text{confirmed}} \subseteq \text{retrieveVC}(\mathbf{ca}_{\text{final}})$

Operation:

```

1:  $L_{\text{extractableVCC}} = \{\}$ 
2: for  $i \in [0, \text{N}_{\text{CA},\text{final}})$  do
3:    $\text{finalConfirmation}_i \leftarrow \text{VerifyExtractableVCCConfirmationAttempt}$ 
   (Inputs for confirmation attempt  $\mathbf{ca}_{\text{final},i}$ )  $\triangleright$  See Algorithm 4.4
4:   if  $\text{finalConfirmation}_i = \top$  then
5:      $L_{\text{extractableVCC}} \leftarrow L_{\text{extractableVCC}} \cup \mathbf{vc}_i$   $\triangleright$  Add the  $\mathbf{vc}_i$  to the list
6:   end if
7: end for
8: if  $(L_{\text{extractableVCC}} = \mathbf{vc}_{\text{confirmed}})$  then  $\triangleright$  Independent of the order
9:   return  $\top$ 
10: else
11:   return  $\perp$ 
12: end if

```

Output:

The result of the verification: $\top$ if the list of votes with a successful confirmation is correct, $\perp$ otherwise.

Finally, the verifier checks that the list of successful votes correspond to the mix net's initial payload $\mathbf{c}_{\text{dec},0}$. To this end, we define a local function **retrieveVote** that returns the vector of encrypted votes from a vector of verification card IDs.

Verification 2.26 VerifyMixNetInitialPayload

Input:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{\text{Base16}})^{\text{L}_{\text{ID}}}$
 Mix net initial payload $\mathbf{c}_{\text{dec},0} \in \mathbb{H}_l^{\text{Nc}}$
 Voting server's list of successful votes $\mathbf{vc}_{\text{confirmed}} \in (\mathbb{A}_{\text{Base16}})^{\text{Nc}}$

Operation:

```

1: if (retrieveVote( $\mathbf{vc}_{\text{confirmed}}$ ) =  $\mathbf{c}_{\text{dec},0}$ ) then                                ▷ Independent of the order
2:   return  $\top$ 
3: else
4:   return  $\perp$ 
5: end if
  
```

Output:

The result of the verification: $\top$ if the mix net initial payload $\mathbf{c}_{\text{dec},0}$ corresponds to the encrypted votes with a successful confirmation, $\perp$ otherwise.

4.1 Consistency Verifications

In our threat model, we assume that only one out of four control components is trustworthy. Without having a trusted source of truth, the verifier must check that each control component's view on the relevant data items is consistent. The verifier must ensure this consistency for both the **SendVote** and **ConfirmVote** algorithm. In this context, consistency is defined as the strict value equality accross control components.

4.1.1 SendVote Consistency

The verifier checks that each control component processed the same set of verification card IDs. During the **DecryptPCC_j** algorithm, the control components check each other's zero-knowledge proofs whose statements already contain numerous data items:

- encrypted vote **E1**
- exponentiated encrypted vote $\widetilde{\mathbf{E1}}$
- encrypted partial Choice Return Codes **E2**
- election public key $\mathbf{EL}_{\text{pk}}$
- verification card public key $\mathbf{K}_{\text{id}}$
- Choice Return Codes encryption public key $\mathbf{pk}_{\text{CCR}}$

However, the verifier must ascertain the consistency of the data objects indicated in table 3.

Verification	Data Objects
2.41 CheckContextConsistency	(p, q, g) , ee , vcs
2.42 CheckPCCConsistency	hpCC_{id} , vc_{id}

Tab. 3: The verifier checks the consistency of the following data objects for every sent vote.

Importantly, the verifier relies on the Return Codes Mapping table **CMtable** and the Voter Choice Return Code Generation public keys $\mathbf{K}_j$ that were used in verification 1.21 and received by the trustworthy setup component.

4.1.2 ConfirmVote Consistency

The verifier validates the confirmation attempts' consistency. Every sent vote might have zero to five corresponding confirmation attempts: depending on the number of times the voter (or, potentially, the adversary) tried to confirm her vote. During the **CreateLVCCShare_j**, the control components check that every confirmation attempt corresponds to a sent vote. Therefore, the verifier must do the following consistency checks as described in table 4.

Verification	Data Objects
2.41 CheckContextConsistency	(p, q, g) , ee , vcs
2.43 CheckCKConsistency	hCK_{id} , attempts_{id} , vc_{id}

Tab. 4: The verifier checks the consistency of the following data objects for every confirmation attempt.

Similar to the **SendVote** verifications, the verifier relies on the Return Codes Mapping table **CMtable** and the Voter Vote Cast Return Code Generation public keys $\mathbf{Kc}_j$ that were used in verification 1.22 and received by the trustworthy setup component.

4.2 Supporting Algorithms

The verifications in block 2 rely on the following algorithms.

Algorithm 4.1 VerifyPCCExponentiationProofsVC

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
 Group generator $g \in \mathbb{G}_q$
 Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{LID}$

Input:

Verification Card ID $\mathbf{vc}_{id} \in (\mathbb{A}_{Base16})^{LID}$
 A vector of hashed, squared partial Choice Return Codes $\mathbf{hpCC}_{id} \in (\mathbb{G}_q)^\psi$
 CCR_j's long Choice Return Code shares $(\mathbf{lCC}_{1,id}, \mathbf{lCC}_{2,id}, \mathbf{lCC}_{3,id}, \mathbf{lCC}_{4,id}) \in (\mathbb{G}_q^\psi)^4$
 Voter Choice Return Code Generation public keys $(K_{1,id}, K_{2,id}, K_{3,id}, K_{4,id}) \in (\mathbb{G}_q)^4$
 Exponentiation proofs $(\pi_{\text{expLCC},1,id}, \pi_{\text{expLCC},2,id}, \pi_{\text{expLCC},3,id}, \pi_{\text{expLCC},4,id}) \in (\mathbb{Z}_q \times \mathbb{Z}_q)^4$

Operation:

```

1:  $\mathbf{g} \leftarrow (g, \mathbf{hpCC}_{id})$  ▷ The bases
2: for  $j \in [1, 4]$  do
3:    $\mathbf{y} \leftarrow (K_{j,id}, \mathbf{lCC}_{j,id})$  ▷ The exponentiations
4:    $\mathbf{i}_{aux} \leftarrow (\mathbf{ee}, \mathbf{vc}_{id}, \text{"CreateLCCShare"}, \text{IntegerToString}(j))$ 
5:    $\text{exponentiationVerif}_j \leftarrow \text{VerifyExponentiation}(\mathbf{g}, \mathbf{y}, \pi_{\text{expLCC},j,id}, \mathbf{i}_{aux})$  ▷ See crypto
   primitives specification
6: end for
7: if  $\text{exponentiationVerif}_j \forall \{j\}$  then
8:   return  $\top$ 
9: else
10:  return  $\perp$ 
11: end if

```

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* verification card, $\perp$ otherwise.

Algorithm 4.2 VerifyExtractableCCVerificationCard

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
 Group generator $g \in \mathbb{G}_q$
 Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Input:

Verification Card ID $\mathbf{vc}_{id} \in (\mathbb{A}_{Base16})^{L_{ID}}$
 CCR_j's long Choice Return Code shares $(\mathbf{lCC}_{1,id}, \mathbf{lCC}_{2,id}, \mathbf{lCC}_{3,id}, \mathbf{lCC}_{4,id}) \in (\mathbb{G}_q^\psi)^4$
 Return Codes Mapping table $\mathbf{CMtable} \in (\mathbb{A}_{Base64}^{1_{HB64}} \times \mathbb{A}_{Base64}^*)^{N_E \cdot (n+1)}$

Operation:

```

1: for  $i \in [0, \psi)$  do
2:    $\mathbf{pC}_{id,i} \leftarrow \prod_{j=1}^4 \mathbf{lCC}_{j,id,i} \bmod p$ 
3:    $\mathbf{lCC}_{id,i} \leftarrow \text{RecursiveHash}(\mathbf{pC}_{id,i}, \mathbf{vc}_{id}, \mathbf{ee}, \text{GetCorrectnessInformationForSelectionIndex}(i))$ 
    $\triangleright$  See crypto primitives specification
4:   if  $\text{Base64Encode}(\text{RecursiveHash}(\mathbf{lCC}_{id,i})) \in \mathbf{CMtable}$  then            $\triangleright$  See crypto primitives
   specification
5:      $\text{extractableIndividualCC}_i \leftarrow \top$ 
6:   else
7:      $\text{extractableIndividualCC}_i \leftarrow \perp$ 
8:   end if
9: end for
10: if  $\text{extractableIndividualCC}_i \forall \{i\}$  then
11:   return  $\top$ 
12: else
13:   return  $\perp$ 
14: end if

```

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* verification card for *all* Choice Return Codes, $\perp$ otherwise.

Algorithm 4.3 VerifyCKExponentiationProofsConfirmationAttempt

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
 Group generator $g \in \mathbb{G}_q$
 Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{LID}$
 Verification Card ID $\mathbf{vc}_{id} \in (\mathbb{A}_{Base16})^{LID}$
 Confirmation attempt $\mathbf{attempts}_{id} \in [0, 5)$

Input:

Hashed, squared confirmation key $\mathbf{hCK}_{id} \in \mathbb{G}_q$
 CCR_j's long Vote Cast Return Code shares $(\mathbf{1VCC}_{id,1}, \mathbf{1VCC}_{id,2}, \mathbf{1VCC}_{id,3}, \mathbf{1VCC}_{id,4}) \in (\mathbb{G}_q)^4$
 Voter Vote Cast Return Code Generation public keys $(\mathbf{Kc}_{1,id}, \mathbf{Kc}_{2,id}, \mathbf{Kc}_{3,id}, \mathbf{Kc}_{4,id}) \in (\mathbb{G}_q)^4$
 Exponentiation proofs $(\pi_{\text{expLVCC},1,id}, \pi_{\text{expLVCC},2,id}, \pi_{\text{expLVCC},3,id}, \pi_{\text{expLVCC},4,id}) \in (\mathbb{Z}_q \times \mathbb{Z}_q)^4$

Operation:

```

1:  $\mathbf{g} \leftarrow (g, \mathbf{hCK}_{id})$  ▷ The bases
2: for  $j \in [1, 4]$  do
3:    $\mathbf{y} \leftarrow (\mathbf{Kc}_{j,id}, \mathbf{1VCC}_{id,j})$  ▷ The exponentiations
4:    $\mathbf{i}_{aux} \leftarrow (\mathbf{ee}, \mathbf{vc}_{id}, \text{"CreateLVCCShare"}, \text{IntegerToString}(j), \text{IntegerToString}(\mathbf{attempts}_{id}))$ 
5:    $\text{exponentiationVerif}_j \leftarrow \text{VerifyExponentiation}(\mathbf{g}, \mathbf{y}, \pi_{\text{expLVCC},j,id}, \mathbf{i}_{aux})$  ▷ See crypto
   primitives specification
6: end for
7: if  $\text{exponentiationVerif}_j \forall \{j\}$  then
8:   return  $\top$ 
9: else
10:  return  $\perp$ 
11: end if

```

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* confirmation attempt, $\perp$ otherwise.

Algorithm 4.4 VerifyExtractableVCCConfirmationAttempt

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
 Group generator $g \in \mathbb{G}_q$
 Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Input:

Verification Card ID $\mathbf{vc}_{id} \in (\mathbb{A}_{Base16})^{L_{ID}}$
 CCR_j's long Vote Cast Return Code shares $(1VCC_{id,1}, 1VCC_{id,2}, 1VCC_{id,3}, 1VCC_{id,4}) \in (\mathbb{G}_q)^4$
 Return Codes Mapping table $\mathbf{CMtable} \in (\mathbb{A}_{Base64}^{1_{HB64}} \times \mathbb{A}_{Base64}^*)^{N_E \cdot (n+1)}$

Operation:

```

1:  $\mathbf{pVCC}_{id} \leftarrow \prod_{j=1}^4 1VCC_{id,j} \mod p$ 
2:  $1VCC_{id} \leftarrow \text{RecursiveHash}(\mathbf{pVCC}_{id}, \mathbf{vc}_{id}, \mathbf{ee})$  ▷ See crypto primitives specification
3: if Base64Encode(RecursiveHash( $1VCC_{id}$ ))  $\in \mathbf{CMtable}$  then ▷ See crypto primitives specification
4:   return  $\top$ 
5: else
6:   return  $\perp$ 
7: end if

```

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

4.3 Secure Logs

Along with the results of their computations and the cryptographic proofs and arguments, the control components also provide a trace of their activity, with the intent of increasing the difficulty of manipulating computations. For this purpose, we implement Scytl’s patented method of generating immutable audit logs (secure logs) [1]. This section explains how the verifier can check the secure logs’ integrity and authenticity.

To inhibit deletion, manipulation or addition of log lines, the logs include a HMAC of the data logged concatenated with the previous HMAC value, thus effectively establishing a chain of logs. At the end of the voting period, the control components should enter a state in which they refuse any new queries, and mark this change of state with a dedicated entry, as a protection against log tail suppression. Alternatively, the auditors can manually check that the log tails are complete.

The keys used for the HMAC are changed regularly, and protected by asymmetric keys. Regular so-called checkpoint entries include the plaintext value of the previous HMAC key, the encrypted value of the new HMAC key and a signature of the checkpoint message. The HMAC of the checkpoint message is included in the signed data.

The secure log entries include the message being logged, along with several additional properties, appended to the end of the line. The properties are wrapped in a block starting with the prefix {*, and ending with the suffix *}. The specific properties included depend on the type of the log entry. They are given in the form `key::value` and are separated by commas.

We distinguish the three following types of log entries:

- Regular log entry: generated by a log request, contains the message, plus the following properties:
 - TS: the timestamp of the log entry, given as the UNIX epoch in milliseconds,
 - HMAC: the HMAC of the log entry, encoded in Base64.
- Checkpoint entry: generated at regular intervals (maximal number of lines or maximal time interval between checkpoints), contains a fixed message, and the following list of properties:
 - LSK: the *liberated* session key, i.e. the key used in the previous block, encoded in Base64,
 - ESK: the new session key, encrypted, encoded in Base64,
 - PHMAC: the HMAC of the previous log entry, as entered in the previous entry’s HMAC property, encoded in Base64,
 - LS: the maximal number of lines allowed between checkpoints,
 - TL: the maximal duration between checkpoints, given in milliseconds,
 - TS: the timestamp of the log entry’s creation time, given as the UNIX epoch in milliseconds,
 - HMAC: a HMAC of the log entry, encoded in Base64,
 - SG: the signature of the log entry, according to algorithm 4.7, encoded in Base64.

- First line entry: generated upon creating the file, contains a fixed message, and the same properties as a regular checkpoint entry, with the exception of LSK and PHMAC: since there is no previous session key, the properties are omitted.

From this it follows that during verification, the verifier parses each line, using the meta-data delimiters provided above. The regular lines in-between checkpoints are considered part of a same block, and their HMAC value verified based on the LSK property of the following checkpoint. The verifier checks the HMAC values and signatures of each checkpoint as well.

Each line's PHMAC property must be strictly equal to the previous line's HMAC value.

Verification 2.51 VerifySecureLogs: Verify the control components' secure logs

Input:

Election event ID $ee \in (\mathbb{A}_{Base16})^{L_{ID}}$

Secure Logs of all control components $L_{SL,1}, L_{SL,2}, L_{SL,3}, L_{SL,4}$

Control components' log signing public keys RSA verification key $K_{sig,1}, K_{sig,2}, K_{sig,3}, K_{sig,4}$

Operation:

```

1: for  $j \in [1, 4]$  do
2:    $i = 0$ 
3:   for each log entry do
4:     if the log entry is a checkpoint then
5:        $\text{logVerif}_{j,i} \leftarrow \text{VerifyLogHmacCheckpoint} \wedge \text{VerifyCheckpointSignature}(K_{sig,j})$   $\triangleright$  See
       algorithm 4.6 and 4.7, the other relevant inputs parameters are implicit
6:     else  $\triangleright$  Corresponds to a regular log
7:        $\text{logVerif}_{j,i} \leftarrow \text{VerifyLogHmac}$   $\triangleright$  See algorithm 4.5, input parameters implicit
8:     end if
9:      $i = i + 1$ 
10:  end for
11: end for
12: if  $\text{logVerif}_{j,i} \forall \{j, i\}$  then
13:  return  $\top$ 
14: else
15:  return  $\perp$ 
16: end if

```

Output:

The result of the verification: $\top$ if the verification is successful for *all* secure log entries, $\perp$ otherwise.

Algorithm 4.5 VerifyLogHmac: Verify the HMAC for a regular log entry

Input:

The current session key $k \in \mathcal{B}^{32}$

The HMAC value of the previous entry $h_{-1} \in \mathcal{B}^{32}$

The timestamp of the log entry $t \in \mathbb{N}$

The log message $m \in (\mathbb{A}_{UCS} \setminus CR, LF)^*$ $\triangleright$ The log message may not contain new line characters

Logged HMAC value $h \in \mathcal{B}^{32}$

Operation:

1: $t_b \leftarrow \text{IntegerToByteArray}(t)$

$\triangleright$ See crypto primitives specification

2: $m_b \leftarrow \text{StringToByteArray}(m)$

$\triangleright$ See crypto primitives specification

3: $h' \leftarrow \text{HMAC}_{\text{SHA-256}}(k, h_{-1} || t_b || m_b)$

4: **if** $h = h'$ **then**

5: **return** $\top$

6: **else**

7: **return** $\perp$

8: **end if**

Output:

$\top$ if the verification succeeds, $\perp$ if it fails.

Algorithm 4.6 VerifyLogHmacCheckpoint: Verify the HMAC for a checkpoint log entry

Input:

- The current session key $k \in \mathcal{B}^{32}$
 - The HMAC value of the previous entry $h_{-1} \in \mathcal{B}^{32}$ $\triangleright$ The empty byte array is used in the case of a first line log entry
 - The liberated session key $k_l \in \mathcal{B}^{32}$
 - The encrypted session key $k_e \in \mathcal{B}^*$
 - The maximal number of log lines between checkpoints $n \in \mathbb{N}$
 - The maximal time duration between checkpoints (in milliseconds) $d \in \mathbb{N}$
 - The timestamp of the log entry $t \in \mathbb{N}$
 - The log message $m \in (\mathbb{A}_{UCS} \setminus CR, LF)^*$ $\triangleright$ The log message may not contain new line characters
 - Logged HMAC value $h \in \mathcal{B}^{32}$
-

Operation:

- 1: $n_b \leftarrow \text{IntegerToByteArray}(n)$ $\triangleright$ See crypto primitives specification
 - 2: $d_b \leftarrow \text{IntegerToByteArray}(d)$
 - 3: $t_b \leftarrow \text{IntegerToByteArray}(t)$
 - 4: $m_b \leftarrow \text{StringToByteArray}(m)$ $\triangleright$ See crypto primitives specification
 - 5: $h' \leftarrow \text{HMAC}_{\text{SHA-256}}(k, h_{-1} || k_l || k_e || n_b || d_b || t_b || m_b)$
 - 6: **if** $h = h'$ **then**
 - 7: **return** $\top$
 - 8: **else**
 - 9: **return** $\perp$
 - 10: **end if**
-

Output:

$\top$ if the verification succeeds, $\perp$ if it fails.

Algorithm 4.7 VerifyCheckpointSignature: Verify the signature for the checkpoint log entry

Context:

- the RSA verification key K_{sig} , defined by:
 - the modulus $m \in \mathbb{P}$
 - the public exponent $p \in \mathbb{N}^*$

Input:

- The HMAC value of the previous entry $h_{-1} \in \mathcal{B}^{32}$ ▷ The empty byte array is used in the case of a first line log entry
 - The liberated session key $k_l \in \mathcal{B}^{32}$
 - The encrypted session key $k_e \in \mathcal{B}^*$
 - The maximal number of log lines between checkpoints $n \in \mathbb{N}$
 - The maximal time duration between checkpoints (in milliseconds) $d \in \mathbb{N}$
 - The timestamp of the log entry $t \in \mathbb{N}$
 - The log message $m \in (\mathbb{A}_{UCS} \setminus CR, LF)^*$
 - The HMAC of the checkpoint log entry $h \in \mathcal{B}^{32}$
 - The signature of the checkpoint entry $s \in \mathcal{B}^*$
-

Operation:

- 1: $n_b \leftarrow \text{IntegerToByteArray}(n)$ ▷ See crypto primitives specification
 - 2: $d_b \leftarrow \text{IntegerToByteArray}(d)$
 - 3: $t_b \leftarrow \text{IntegerToByteArray}(t)$
 - 4: $m_b \leftarrow \text{StringToByteArray}(m)$ ▷ See crypto primitives specification
 - 5: $p \leftarrow h_{-1} || k_l || k_e || n_b || d_b || t_b || m_b || h$
 - 6: $s \leftarrow \text{Verify}_{\text{RSA-PSS}}(K_{\text{sig}}, s, p)$ ▷ Uses standard RSA-PSS signature, with SHA-256 as a digest function.
-

5 Block 3: Mixing Decryption Verification — VerifyOnlineTally

The verifier executes the `VerifyOnlineShuffleProofsBallotBox` and `VerifyOnlineDecryptionProofsBallotBox` algorithm for all ballot boxes. A shuffle requires at least two votes in the ballot box. In practice, the election authorities choose the counting circles as such that there are always a sufficient number of votes in each ballot box. Technically, however, we have to skip the shuffle proof verification if there are less than two votes in a ballot box. Please note that other verifications ensure that each control component shuffles *all* confirmed votes (and only confirmed votes) of a ballot box. The verification succeeds *only* if the validation of *all* ballot boxes succeed.

Verification 3.01 VerifyOnlineShuffleProofs

Input:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Vector of ballot box IDs $\mathbf{bb} = (\mathbf{bb}_0, \dots, \mathbf{bb}_{N_{bb}-1}) \in ((\mathbb{A}_{Base16})^{L_{ID}})^{N_{bb}}$

Operation:

```

1: for  $i \in [0, N_{bb})$  do
2:   if  $\mathbf{bb}_i$  contains more than 1 vote then
3:      $\mathbf{bbShuffleVerif}_i \leftarrow \text{VerifyOnlineShuffleProofsBallotBox}(\text{Inputs for ballot box } \mathbf{bb}_i)$     ▷
     See Algorithm 5.1
4:   else
5:      $\mathbf{bbShuffleVerif}_i \leftarrow \top$                                 ▷ We skip the check if there are less than two votes
6:   end if
7: end for
8: if  $\mathbf{bbShuffleVerif}_i \forall i$  then
9:   return  $\top$ 
10: else
11:   return  $\perp$ 
12: end if
  
```

Output:

The result of the verification: $\top$ if the verification is successful for *all* ballot boxes, $\perp$ otherwise.

Verification 3.02 VerifyOnlineDecryptionProofs

Input:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Vector of ballot box IDs $\mathbf{bb} = (\mathbf{bb}_0, \dots, \mathbf{bb}_{N_{bb}-1}) \in ((\mathbb{A}_{Base16})^{L_{ID}})^{N_{bb}}$

Operation:

```
1: for  $i \in [0, N_{bb})$  do
2:   if  $\mathbf{bb}_i$  contains at least 1 vote then
3:      $\mathbf{bbDecryptVerif}_i \leftarrow \text{VerifyOnlineDecryptionProofsBallotBox}(\text{Inputs for ballot box } \mathbf{bb}_i)$ 
    ▷ See Algorithm 5.2
4:   else
5:      $\mathbf{bbDecryptVerif}_i \leftarrow \top$                                 ▷ We skip the check if there is no vote
6:   end if
7: end for
8: if  $\mathbf{bbDecryptVerif}_i \forall \{i\}$  then
9:   return  $\top$ 
10: else
11:   return  $\perp$ 
12: end if
```

Output:

The result of the verification: $\top$ if the verification is successful for *all* ballot boxes, $\perp$ otherwise.

5.1 Supporting Algorithms

The verifications in block 3 rely on the following algorithms.

Algorithm 5.1 VerifyOnlineShuffleProofsBallotBox

Context:

Group modulus $p \in \mathbb{P}$
Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
Group generator $g \in \mathbb{G}_q$
Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$
Ballot box ID $\mathbf{bb} \in (\mathbb{A}_{Base16})^{L_{ID}}$ $\triangleright$ The ID of the ballot box being verified
Number of allowed write-ins + 1 for this specific ballot box $\hat{\delta} \in \mathbb{N}^*$

Input:

Partially decrypted votes $(\mathbf{c}_{dec,0}, \mathbf{c}_{dec,1}, \mathbf{c}_{dec,2}) \in (\mathbb{H}_l^{N_c})^3$
Remaining election public keys $(\overline{\mathbf{EL}}_{pk,0}, \overline{\mathbf{EL}}_{pk,1}, \overline{\mathbf{EL}}_{pk,2}) \in (\mathbb{G}_q^\delta)^3$
Shuffled votes $(\mathbf{c}_{mix,1}, \mathbf{c}_{mix,2}, \mathbf{c}_{mix,3}) \in (\mathbb{H}_l^{N_c})^3$
Shuffle arguments $(\pi_{mix,1}, \pi_{mix,2}, \pi_{mix,3})$ $\triangleright$ See the domain of the Shuffle argument

Ensure: $l = \hat{\delta}$

Ensure: $N_c \geq 2$ $\triangleright$ The algorithm requires at least two votes

Ensure: $0 < l \leq \delta$

Operation:

```

1: for  $j \in [1, 3]$  do
2:    $\text{shuffleVerif}_j \leftarrow \text{VerifyShuffle}(\mathbf{c}_{dec,j-1}, \mathbf{c}_{mix,j}, \pi_{mix,j}, \overline{\mathbf{EL}}_{pk,j})$   $\triangleright$  See crypto primitives
   specification
3: end for
4: if  $\text{shuffleVerif}_1 \wedge \text{shuffleVerif}_2 \wedge \text{shuffleVerif}_3$  then
5:   return  $\top$ 
6: else
7:   return  $\perp$ 
8: end if

```

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* ballot box,
 $\perp$ otherwise.

Algorithm 5.2 VerifyOnlineDecryptionProofsBallotBox

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
 Group generator $g \in \mathbb{G}_q$
 Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$
 Ballot box ID $\mathbf{bb} \in (\mathbb{A}_{Base16})^{L_{ID}}$ $\triangleright$ The ID of the ballot box being verified
 Number of allowed write-ins + 1 for this specific ballot box $\hat{\delta} \in \mathbb{N}^*$

Input:

Encrypted votes $(\mathbf{c}_{\text{mix},1}, \mathbf{c}_{\text{mix},2}, \mathbf{c}_{\text{mix},3}) \in (\mathbb{H}_l^{N_c})^3$
 CCM_j election public keys $(\mathbf{EL}_{\text{pk},1}, \mathbf{EL}_{\text{pk},2}, \mathbf{EL}_{\text{pk},3}) \in (\mathbb{G}_q^\mu)^3$
 Partially decrypted votes $(\mathbf{c}_{\text{dec},1}, \mathbf{c}_{\text{dec},2}, \mathbf{c}_{\text{dec},3}) \in (\mathbb{H}_l^{N_c})^3$
 Decryption proofs $(\boldsymbol{\pi}_{\text{dec},1}, \boldsymbol{\pi}_{\text{dec},2}, \boldsymbol{\pi}_{\text{dec},3}) \in ((\mathbb{Z}_q \times \mathbb{Z}_q^l)^{N_c})^3$

Ensure: $l = \hat{\delta}$

Ensure: $N_c \geq 1$

$\triangleright$ The algorithm requires at least one vote

Ensure: $0 < l \leq \mu$

Operation:

```

1: for  $j \in [1, 3]$  do
2:    $\mathbf{i}_{\text{aux}} \leftarrow (\mathbf{ee}, \mathbf{bb}, \text{"MixDecOnline"}, \text{IntegerToString}(j))$ 
3:    $\text{decryptVerif}_j \leftarrow \text{VerifyDecryptions}(\mathbf{c}_{\text{mix},j}, \mathbf{EL}_{\text{pk},j}, \mathbf{c}_{\text{dec},j}, \boldsymbol{\pi}_{\text{dec},j}, \mathbf{i}_{\text{aux}})$   $\triangleright$  See crypto
    primitives specification
4: end for
5: if  $\text{decryptVerif}_j \forall \{j\}$  then
6:   return  $\top$ 
7: else
8:   return  $\perp$ 
9: end if

```

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* ballot box, $\perp$ otherwise.

5.2 Block 3: Authenticity Verifications

The verifier checks the authenticity of the following data objects.

Verification	Message	Sign Key
3.71 CheckSigOnlineProofs	$\mathbf{c}_{\text{mix},1}, \mathbf{c}_{\text{dec},1}, \pi_{\text{mix},1}, \pi_{\text{dec},1}, \overline{\text{EL}}_{\text{pk},0}, \overline{\text{EL}}_{\text{pk},1}, \text{EL}_{\text{pk},1}$	$\text{CCR}_{\text{sk},1}$
	$\mathbf{c}_{\text{mix},2}, \mathbf{c}_{\text{dec},2}, \pi_{\text{mix},2}, \pi_{\text{dec},2}, \overline{\text{EL}}_{\text{pk},1}, \overline{\text{EL}}_{\text{pk},2}, \text{EL}_{\text{pk},2}$	$\text{CCR}_{\text{sk},2}$
	$\mathbf{c}_{\text{mix},3}, \mathbf{c}_{\text{dec},3}, \pi_{\text{mix},3}, \pi_{\text{dec},3}, \overline{\text{EL}}_{\text{pk},2}, \overline{\text{EL}}_{\text{pk},3}, \text{EL}_{\text{pk},3}$	$\text{CCR}_{\text{sk},3}$

Tab. 5: The verifier checks the following data objects in Block 3 (VerifyOnlineTally). The correctness of the initial list of ciphertexts $\mathbf{c}_{\text{dec},0}$ is checked in Block 2.

6 Block 4: Result Verification — VerifyOfflineTally

Similarly to block 3, the verifier checks the shuffle and decryption proofs for all ballot boxes. Again, the case where there is zero or one vote in a ballot box is technically possible but does not arise in practice. Other verifications ensure that the offline control component mixes and decrypts *all* confirmed votes of a ballot box.

Verification 4.11 VerifyOfflineShuffleProof

Input:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Vector of ballot box IDs $\mathbf{bb} = (\mathbf{bb}_0, \dots, \mathbf{bb}_{N_{bb}-1}) \in ((\mathbb{A}_{Base16})^{L_{ID}})^{N_{bb}}$

Operation:

```

1: for  $i \in [0, N_{bb})$  do
2:   if  $\mathbf{bb}_i$  contains more than 1 vote then
3:      $\mathbf{bbShuffleVerif}_i \leftarrow \text{VerifyOfflineShuffleProofBallotBox}(\text{Inputs for ballot box } \mathbf{bb}_i)$   $\triangleright$  See
       Algorithm 6.1
4:   else
5:      $\mathbf{bbShuffleVerif}_i \leftarrow \top$   $\triangleright$  We skip the check if there are less than two votes
6:   end if
7: end for
8: if  $\mathbf{bbShuffleVerif}_i \forall i$  then
9:   return  $\top$ 
10: else
11:   return  $\perp$ 
12: end if
```

Output:

The result of the verification: $\top$ if the verification is successful for *all* ballot boxes, $\perp$ otherwise.

Verification 4.12 VerifyOfflineDecryptionProofs

Input:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Vector of ballot box IDs $\mathbf{bb} = (\mathbf{bb}_0, \dots, \mathbf{bb}_{N_{bb}-1}) \in ((\mathbb{A}_{Base16})^{L_{ID}})^{N_{bb}}$

Operation:

```
1: for  $i \in [0, N_{bb})$  do
2:   if  $\mathbf{bb}_i$  contains at least 1 vote then
3:      $\mathbf{bbDecryptVerif}_i \leftarrow \text{VerifyOfflineDecryptionProofsBallotBox}(\text{Inputs for ballot box } \mathbf{bb}_i)$ 
    ▷ See Algorithm 6.2
4:   else
5:      $\mathbf{bbDecryptVerif}_i \leftarrow \top$                                 ▷ We skip the check if there is no vote
6:   end if
7: end for
8: if  $\mathbf{bbDecryptVerif}_i \forall \{i\}$  then
9:   return  $\top$ 
10: else
11:   return  $\perp$ 
12: end if
```

Output:

The result of the verification: $\top$ if the verification is successful for *all* ballot boxes, $\perp$ otherwise.

Ultimately, the verifier checks the decoding of the decrypted votes $\mathbf{c}_{\text{dec},4}$ by validating the last control components list of decoded votes L_{votes} .

Verification 4.13 VerifyPlaintextsDecoding

Input:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Vector of ballot box IDs $\mathbf{bb} = (\mathbf{bb}_0, \dots, \mathbf{bb}_{N_{bb}-1}) \in ((\mathbb{A}_{Base16})^{L_{ID}})^{N_{bb}}$

Operation:

```

1: for  $i \in [0, N_{bb})$  do
2:   if  $\mathbf{bb}_i$  contains at least 1 vote then
3:      $\mathbf{bbDecodeVerif}_i \leftarrow \text{VerifyPlaintextsDecodingBallotBox}(\text{Inputs for ballot box } \mathbf{bb}_i)$     ▷
     See Algorithm 6.3
4:   else
5:      $\mathbf{bbDecodeVerif}_i \leftarrow \top$                                 ▷ We skip the check if there is no vote
6:   end if
7: end for
8: if  $\mathbf{bbDecodeVerif}_i \forall \{i\}$  then
9:   return  $\top$ 
10: else
11:   return  $\perp$ 
12: end if
  
```

Output:

The result of the verification: $\top$ if the verification is successful for *all* ballot boxes, $\perp$ otherwise.

6.1 Supporting Algorithms

The verifications in block 4 rely on the following algorithms.

Algorithm 6.1 VerifyOfflineShuffleProofBallotBox

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
 Group generator $g \in \mathbb{G}_q$
 Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$
 Ballot box ID $\mathbf{bb} \in (\mathbb{A}_{Base16})^{L_{ID}}$
 Number of allowed write-ins + 1 for this specific ballot box $\hat{\delta} \in \mathbb{N}^*$

Input:

Partially decrypted votes $\mathbf{c}_{dec,3} \in \mathbb{H}_l^{N_c}$
 Electoral board public key $\mathbf{EB}_{pk} \in \mathbb{G}_q^\delta$
 Shuffled votes $\mathbf{c}_{mix,4} \in \mathbb{H}_l^{N_c}$
 Shuffle proof $\pi_{mix,4}$

▷ See the domain of the Shuffle proof

Ensure: $l = \hat{\delta}$

Ensure: $N_c \geq 2$

▷ The algorithm requires at least two votes

Ensure: $0 < l \leq \delta$

Operation:

```

1: shuffleVerif4 ← VerifyShuffle( $\mathbf{c}_{dec,3}$ ,  $\mathbf{c}_{mix,4}$ ,  $\pi_{mix,4}$ ,  $\overline{\mathbf{EL}}_{pk,3}$ ) ▷ See crypto primitives specification
2: if shuffleVerif4 then
3:   return  $\top$ 
4: else
5:   return  $\perp$ 
6: end if
    
```

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* ballot box, $\perp$ otherwise.

Algorithm 6.2 VerifyOfflineDecryptionProofsBallotBox

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
 Group generator $g \in \mathbb{G}_q$
 Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$
 Ballot box ID $\mathbf{bb} \in (\mathbb{A}_{Base16})^{L_{ID}}$
 Number of allowed write-ins + 1 for this specific ballot box $\hat{\delta} \in \mathbb{N}^*$
 Maximum number of allowed write-ins + 1 for this election event $\delta \in \mathbb{N}^*$
 Maximum supported number of write-ins + 1 $\mu \in \mathbb{N}^*$

Input:

Encrypted votes $\mathbf{c}_{mix,4} = (c_{mix,4,0}, \dots, c_{mix,4,N_c-1}) \in \mathbb{H}_l^{N_c}$
 Electoral board public key $\mathbf{EB}_{pk} \in \mathbb{G}_q^\delta$
 Decrypted votes $\mathbf{c}_{dec,4} \in \mathbb{H}_l^{N_c}$
 Decryption proofs $\boldsymbol{\pi}_{dec,4} = (\pi_{dec,4,0}, \dots, \pi_{dec,4,N_c-1}) \in (\mathbb{Z}_q \times \mathbb{Z}_q^l)^{N_c}$

Ensure: $l = \hat{\delta}$

Ensure: $N_c \geq 1$

▷ The algorithm requires at least one vote

Ensure: $0 < l \leq \delta \leq \mu$

Operation:

```

1:  $\mathbf{i}_{aux} \leftarrow (\mathbf{ee}, \mathbf{bb}, \text{"MixDecOffline"})$ 
2:  $\text{decryptVerif}_4 \leftarrow \text{VerifyDecryptions}(\mathbf{c}_{mix,4}, \mathbf{EB}_{pk}, \mathbf{c}_{dec,4}, \boldsymbol{\pi}_{dec,4}, \mathbf{i}_{aux})$     ▷ See crypto primitives
   specification
3: if  $\text{decryptVerif}_4$  then
4:   return  $\top$ 
5: else
6:   return  $\perp$ 
7: end if

```

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* ballot box,
 $\perp$ otherwise.

Algorithm 6.3 VerifyPlaintextsDecodingBallotBox

Context:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Ballot box ID $\mathbf{bb} \in (\mathbb{A}_{Base16})^{L_{ID}}$

Input:

List of decoded votes $L_{votes} = (\hat{\mathbf{v}}_0, \dots, \hat{\mathbf{v}}_{N_C-1})$

Decrypted votes $\mathbf{c}_{dec,4} \in \mathbb{H}_l^{N_C}$

The primes mapping table $\mathbf{pTable} = (\tilde{\mathbf{p}}, \tilde{\mathbf{v}})$ $\triangleright$ Received after the configuration phase

Number of selectable voting options $\psi \in [1, 120]$

Operation:

```

1: for  $i \in [0, N_C)$  do
2:    $\mathbf{c}_{dec,4,i} = (\gamma_i, \phi_{i,0}, \dots, \phi_{i,l-1})$ 
3:    $\mathbf{m}_i \leftarrow (\phi_{i,0}, \dots, \phi_{i,l-1})$ 
4: end for
5:  $\mathbf{m} \leftarrow (m_0, \dots, m_{N_C-1})$ 
6:  $L'_{votes} \leftarrow \text{DecodePlaintexts}(\mathbf{m}, \psi, \mathbf{pTable})$   $\triangleright$  See system specification
7: if  $(L_{votes} = L'_{votes})$  then
8:   return  $\top$ 
9: else
10:  return  $\perp$ 
11: end if

```

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* ballot box, $\perp$ otherwise.

6.2 Block 4: Authenticity Verifications

Verification	Message	Sign Key
4.71 CheckSigOfflineTally	$\mathbf{c}_{mix,4}, \mathbf{m}, \pi_{mix,4}, \pi_{dec,4}, \mathbf{EB}_{pk}, L_{votes}$	$\mathbf{AB}_{sk}$

Tab. 6: The verifier checks the following data objects in Block 4 (VerifyOfflineTally). The correctness of the input list of ciphertexts $\mathbf{c}_{dec,3}$ is checked in Block 3.

References

- [1] A. O. Cornet and J. M. B. Bosch: *Method and system of generating immutable audit logs*. US Patent 8,422,682. 2006. URL: <https://patents.google.com/patent/EP2019992A1>.
- [2] Die Schweizerische Bundeskanzlei (BK): *Federal Chancellery Ordinance on Electronic Voting (OEV), Draft of 28 April 2021*.
- [3] Die Schweizerische Bundeskanzlei (BK): *Partial revision of the Ordinance on Political Rights and total revision of the Federal Chancellery Ordinance on Electronic Voting. Redesign of Trials. Explanatory report for consultation*. 2021.
- [4] Eidgenössisches Departement für auswärtige Angelegenheiten EDA: *Swiss Political System - Direct Democracy*. <https://www.eda.admin.ch/aboutswitzerland/en/home/politik/uebersicht/direkte-demokratie.html/>. Retrieved on 2020-07-15.
- [5] gfs.bern: *Vorsichtige Offenheit im Bereich digitale Partizipation - Schlussbericht*. Mar. 2020.
- [6] R. Haenni: *Swiss Post Public Intrusion Test: Undetectable attack against vote integrity and secrecy*. 2019.
- [7] R. Haenni, E. Dubuis, R. E. Koenig, and P. Locher: “CHVote: Sixteen Best Practices and Lessons Learned”. In: *International Joint Conference on Electronic Voting*. Springer. 2020, pp. 95–111.
- [8] R. Haenni, E. Dubuis, R. E. Koenig, and P. Locher: “Process models for universally verifiable elections”. In: *International Joint Conference on Electronic Voting*. Springer. 2018, pp. 84–99.
- [9] T. Haines, S. J. Lewis, O. Pereira, and V. Teague: “How not to prove your election outcome”. In: *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE. 2020, pp. 644–660.
- [10] S. Josefsson et al.: *The base16, base32, and base64 data encodings*. Tech. rep. RFC 4648, October, 2006.
- [11] P. Locher, R. Haenni, and R. E. Koenig: *Analysis of the cryptographic implementation of the swiss post voting protocol*. 2019.
- [12] S. Post: *Protocol of the Swiss Post Voting System. Computational Proof of Complete Verifiability and Privacy. Version 0.9.10*. <https://gitlab.com/swisspost-evoting/e-voting/e-voting-documentation/-/tree/master/Protocol>. June 2021.
- [13] B. Smyth: “A foundation for secret, verifiable elections.” In: *IACR Cryptology ePrint Archive* 2018 (2018), p. 225.
- [14] V. Teague and O. Pereira: *Report on the SwissPost-Scytl e-voting system, trusted-server version*. 2019.

List of Algorithms

3.1	VerifyEncryptedPCCExponentiationProofsVerificationCardSet	18
3.2	VerifyEncryptedCKExponentiationProofsVerificationCardSet	19
4.1	VerifyPCCExponentiationProofsVC	27
4.2	VerifyExtractableCCVerificationCard	28
4.3	VerifyCKExponentiationProofsConfirmationAttempt	29
4.4	VerifyExtractableVCCConfirmationAttempt	30
4.5	VerifyLogHmac: Verify the HMAC for a regular log entry	33
4.6	VerifyLogHmacCheckpoint: Verify the HMAC for a checkpoint log entry	34
4.7	VerifyCheckpointSignature: Verify the signature for the checkpoint log entry	35
5.1	VerifyOnlineShuffleProofsBallotBox	38
5.2	VerifyOnlineDecryptionProofsBallotBox	39
6.1	VerifyOfflineShuffleProofBallotBox	44
6.2	VerifyOfflineDecryptionProofsBallotBox	45
6.3	VerifyPlaintextsDecodingBallotBox	46

List of Verifications

1.01	VerifyEncryptionParameters	13
1.02	VerifySmallPrimeGroupMembers	14
1.03	VerifyVotingOptions	15
1.21	VerifyEncryptedPCCExponentiationProofs	16
1.22	VerifyEncryptedCKExponentiationProofs	17
2.21	VerifyPCCExponentiationProofs	20
2.22	VerifyExtractableCC	21
2.23	VerifyCKExponentiationProofs	22
2.24	VerifyNonFinalConfirmationAttempts	23
2.25	VerifyListOfVotesWithSuccessfulConfirmation	24
2.26	VerifyMixNetInitialPayload	25
2.51	VerifySecureLogs: Verify the control components' secure logs	32
3.01	VerifyOnlineShuffleProofs	36
3.02	VerifyOnlineDecryptionProofs	37
4.11	VerifyOfflineShuffleProof	41
4.12	VerifyOfflineDecryptionProofs	42
4.13	VerifyPlaintextsDecoding	43

List of Figures

1	Overview of the Election Event Context	10
2	Control Component Authenticity Check	12

List of Tables

1	Verification categories and their description.	9
2	Overview of the Verification Blocks.	10
3	Block 2 consistency checks	26
4	Block 2 consistency checks	26
5	Block 3 Authenticity Checks	40
6	Block 4 Authenticity Checks	46